

FROM NOTEBOOKS TO PRODUCTION

WHISPER · DOCKER · EVENT-DRIVEN ARCHITECTURE · PRODUCTION ML

ALAGU PRAKLAYA , MLE@ASAPP <> DOCKER CAPTAIN

What We Thought vs. What Actually Happened

✗ WHAT WE THOUGHT

"A model in a notebook is 80% of the way to production. We just need to wrap it in an API."

✓ WHAT ACTUALLY HAPPENED

The notebook IS the prototype.
Production is a completely different system design problem.

The gap is architectural — not syntactic.

Three Ways Teams Fail Moving to Production

1

MODEL-AS-PRODUCT

You shipped a feature,
not a system. The pipeline
around the model IS the product.

2

HAPPY PATH ONLY

It works on clean demo data.
It breaks on dark photos,
noise, edge cases, real inputs.

3

SINGLE-MACHINE THINKING

Designed for one machine,
one model, one request.
Architecture can't scale.

ACT 2

THE DOCUMENTARY ARCHITECTURE STORIES

NETFLIX

DOORDASH

DISCORD

SPOTIFY

CLOUDFLARE

Metaflow: Closing the Notebook-to-Pipeline Gap

THE PROBLEM

Thousands of DS writing notebooks.
Engineers re-write them as pipelines.
Weeks to reach production.

THE METAFLOW INSIGHT

@step decorator = your contract.

Same Python code runs locally
AND in production.

Experiment $\equiv$ Production Job.

Your Whisper Step Is Your Contract

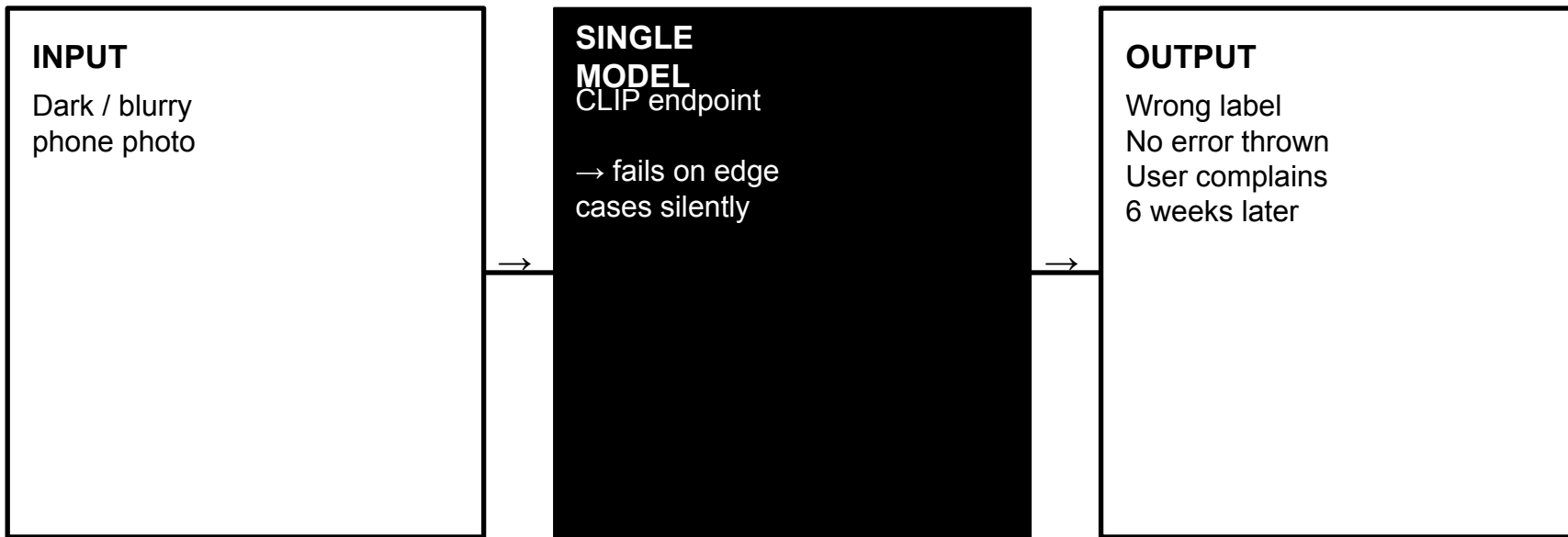
STEAL THIS → PIPELINE THINKING

Your transcription function should be a `@step`, not a script.

- ✓ Takes explicit inputs: (audio_bytes, config)
- ✓ Produces explicit outputs: (transcript_json)
- ✓ Stateless — no global state, no path assumptions
- ✓ Retryable safely

If it touches global state or assumes a specific file path — it's a script that happens to work today.

DashCLIP: Single Endpoint vs. Computational Graph



vs. GRAPH: Audio quality check → Language detect → Whisper → Post-process

Osprey: 10x Throughput on the Same Hardware

6–7M

**CONCURRENT
VOICE STREAMS**

at peak

THE ARCHITECTURE

- Elixir actor model → millions of lightweight processes
- Rust hot path → zero GC pauses, predictable latency
- C++ bindings → optimized inference kernel
- Result: 10x throughput, same hardware budget

Resource Efficiency Is Strategy

NAIVE CONTAINER

`docker run whisper`
→ 8 GB RAM
→ 3 sec cold start
→ model loads per request

OPTIMIZED CONTAINER

`ctranslate2 backend`
→ 1.8 GB RAM
→ model loads once at startup
→ sub-200ms warm inference

Same model. Different runtime. 4× the scale.

From Batch to Real-Time: The PAS Architecture

BATCH (BEFORE)

Calculate recommendations
once per night.

React to what you did
24 hours ago.



EVENT-DRIVEN (NOW)

Every action is an event.
Events are appended to a log.
ML workers subscribe in real-time.

React to the song you just skipped.

P-A-S

PRODUCER · APPENDER · SUBSCRIBER

DDIA Chapter 11 — Stream Processing — In Action

P PRODUCER

Audio API receives file.
Publishes event to queue
with audio ref + metadata.

Returns immediately.

A APPENDER

Redis Streams / Kafka /
SQS — doesn't matter.

Durable. Ordered. Replayable.
Survives crashes.

S SUBSCRIBER

Docker container.
Whisper worker wakes
when event arrives.

Publishes result event.

Pingora: Measure Before You Accept Defaults

NGINX ASSUMPTION

Worker model creates a new connection per request to upstream servers.

'This is fine.'



PINGORA REALITY

Shared connection pool across all threads.

→ 160 billion fewer TCP handshakes per day.

'We measured it.'

Three Numbers You Must Know Before You Ship

01 STARTUP TIME

docker run → first inference
Target: <3 sec
Fix: pre-load model, not lazy-load

02 MEMORY FOOTPRINT

docker stats during inference
Target: <2 GB (Whisper large)
Fix: ctranslate2 + INT8 quant

03 CONCURRENCY

2 files arrive simultaneously
Target: queued, not crashed
Fix: async workers + PAS

LIVE DEMO

Building the PAS Pattern with Whisper

*"What I'm about to show you is not a Whisper tutorial.
Whisper is one line of Python — that part is trivial.*

I'm showing you the architecture AROUND Whisper.

Three Services. One Docker Compose File.

PRODUCER

producer.py

Receives audio path.
Publishes event to
Redis Streams with
audio ref + metadata.

WORKER

worker.py (Whisper)

Subscribes to Redis.
Runs ctranslate2.
INT8 quantized.
Publishes result event.

CONSUMER

consumer.py

Reads result events.
Prints transcript
with metadata:
latency, model version.

Redis Streams (durable, replayable) runs between all three

Five Companies. Five Lessons.

NETFLIX

Experiment $\equiv$ Production. No translation tax.

DOORDASH

A graph, not an endpoint. Route before you infer.

DISCORD

Architecture is a variable. Measure the hot path.

SPOTIFY

Producer never waits for subscriber. PAS first.

CLOUDFLARE

Know your numbers before they ask.

The Gap

NOTEBOOK

```
whisper.transcribe(  
    audio_path  
)
```

```
# done?  
# ship it?
```



PRODUCTION

PAS event architecture
+ Computational graph
+ INT8 quantized runtime
+ Observed, retried, versioned
+ Resource-profiled
+ Replayable job log



The agency didn't fail because of their model.

They failed because they had no architecture around it.

Now you do.

The Reading List

NETFLIX

Engineering Blog — search 'Metaflow'

DOORDASH

Engineering Blog — search 'DashCLIP'

DISCORD

Engineering Blog — search 'Osprey'

SPOTIFY

Engineering Blog — search 'Scio' or 'Event-driven ML'

CLOUDFLARE

Blog — 'How we built Pingora'

BOOKS

DDIA (Kleppmann) Ch. 11 · Designing ML Systems (Huyen) Ch. 7–9